DECA Lab Logbook — Spring Term

Lab 3 | Task 3: Polling I/O

Task 3: Polling I/O

Objectives

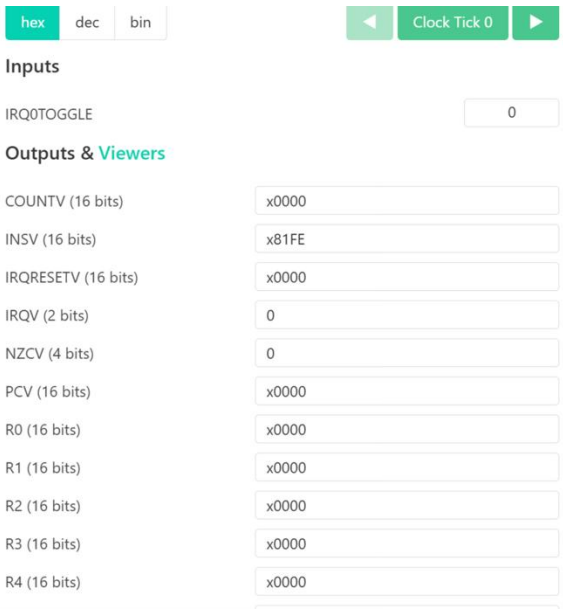
- Understand the software polling technique for detecting hardware events
- Use the step simulator to manually trigger IRQ0 and observe the polling response
- Use the waveform simulator to observe automatic IRQ1 polling

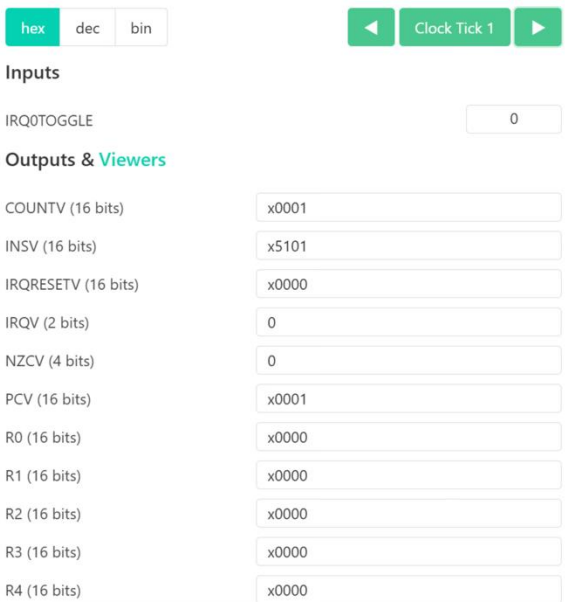
3.1 iotest1: Polling IRQ0 (Step Simulator)

Simulate the eep1 (not eep1x) sheet of lab3i with instruction memory loaded from iotest1.ram in the step simulator. Run a few cycles to observe the normal polling loop. Then manually toggle IRQ0TOGGLE and observe how this generates an IRQ0 request, which the program detects and clears, incrementing R2.

```
LDR R0, [0xFFFE]   // load IRQ status
AND R0, #1          // test IRQ0
JEQ -2              // loop if no IRQ0
MOV R0, #1
STR R0, [0xFFFF]   // reset IRQ0
MOV R0, #0
STR R0, [0xFFFF]   // stop reset
ADD R2, #1          // increment software counter
JMP -8
```

Normal polling behaviour (IRQ0TOGGLE not toggled): IRQ0 = 0, so AND R0, #1 produces zero, JEQ −2 is always taken, and the program loops continuously through LDR → AND → JEQ. R2 remains unchanged and PC cycles with period 3.





hex dec bin ◀ Clock Tick 2 ▶

Inputs

IRQ0TOGGLE	0

Outputs & Viewers

COUNTV (16 bits)	x0002
INSV (16 bits)	xC2FE
IRQRESETV (16 bits)	x0000
IRQV (2 bits)	0
NZCV (4 bits)	4
PCV (16 bits)	x0002
R0 (16 bits)	x0000
R1 (16 bits)	x0000
R2 (16 bits)	x0000
R3 (16 bits)	x0000
R4 (16 bits)	x0000
R5 (16 bits)	x0000

hex dec bin ◀ Clock Tick 3 ▶

Inputs

IRQ0TOGGLE	0

Outputs & Viewers

COUNTV (16 bits)	x0003
INSV (16 bits)	x81FE
IRQRESETV (16 bits)	x0000
IRQV (2 bits)	0
NZCV (4 bits)	4
PCV (16 bits)	x0000
R0 (16 bits)	x0000
R1 (16 bits)	x0000
R2 (16 bits)	x0000
R3 (16 bits)	x0000
R4 (16 bits)	x0000
R5 (16 bits)	x0000

After toggling IRQ0TOGGLE: The external hardware detects the edge on IRQ0TOGGLE and asserts IRQ0 = 1. On the next poll, AND R0, #1 is non-zero, so JEQ is not taken. The program writes 1 then 0 to address 0xFFFF, generating an IRQ0reset pulse that clears the interrupt request. It then increments R2 and jumps back to the start of the loop.

IRQ0TOGGLE was toggled twice during simulation, so R2 reaches 2 and IRQ0TOGGLE returns to its original value. The program therefore counts edges on IRQ0TOGGLE using memory-mapped I/O and a software polling loop.

IRQ0TOGGLE	0

Outputs & Viewers

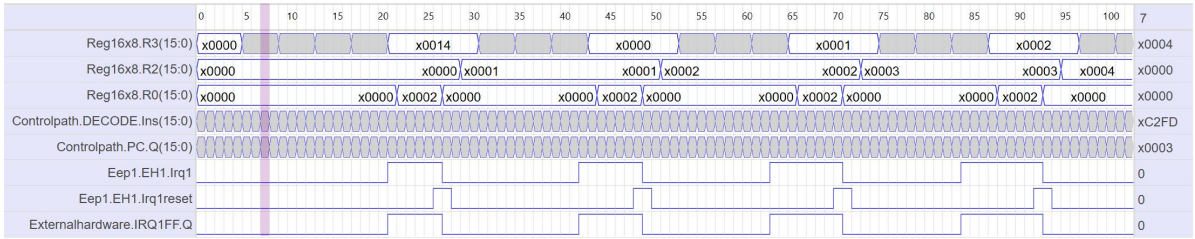
COUNTV (16 bits)	x0002
INSV (16 bits)	xC0F8
IRQRESETV (16 bits)	x0000
IRQV (2 bits)	2
NZCV (4 bits)	0
PCV (16 bits)	x0008
R0 (16 bits)	x0000
R1 (16 bits)	x0000
R2 (16 bits)	x0002
R3 (16 bits)	x0000
R4 (16 bits)	x0000
R5 (16 bits)	x0000

3.2 iotest2: Polling IRQ1 (Waveform Simulator)

Repeat the task using iotest2.ram in the waveform simulator. The program iotest2.txt detects interrupt source 1 (INT1), which repeatedly sets IRQ1 approximately every 20 clock cycles via a counter on the external-hardware sheet. View IRQ1 and IRQ1reset in addition to PC, INS, R0, and R2.

```
LDR R3, [0xFFFF]   // load COUNT
LDR R0, [0xFFFE]   // load IRQ status
AND R0, #2          // test IRQ1
JEQ -3
MOV R0, #2
STR R0, [0xFFFF]   // reset IRQ1
MOV R0, #0
STR R0, [0xFFFF]   // stop reset
ADD R2, #1          // increment software counter
JMP -9
```

IRQ1 is generated automatically by a hardware counter on the external hardware sheet, asserting IRQ1 approximately every 20 clock cycles. In the waveform simulator the following repeating sequence is observed:

- IRQ1 goes high (asserted by hardware counter).
- The polling loop detects bit 1 set; JEQ is not taken.
- The program writes 2 then 0 to 0xFFFF, generating an IRQ1reset pulse and clearing the request.
- R2 increments by 1 and the program returns to polling.

This pattern repeats at approximately 20-cycle intervals. AND R0, #2 isolates bit 1 (IRQ1); bit 0 (IRQ0) is ignored. Each IRQ1 pulse is therefore followed by an IRQ1reset pulse and a one-count increase in R2.